

Hardware Security Analysis Report: Hardware Trojan & VHDL Backdoor Analysis

Rabi Bhushan Yadav

August 31, 2026

1 Executive Summary

This report documents the reverse-engineering and security audit of a 16-bit hardware architecture implemented in VHDL, simulating a compromised Trusted Platform Module (TPM). The circuit embeds a **Hardware Trojan (Backdoor)** that intercepts normal cryptographic functionality and leaks the internal master key to the primary output bus upon receiving a specific 16-bit trigger pattern.

- **Target Input Trigger Code:** 1111111111101001 (0xFFE9 / 65513)
- **Extracted Key Payload:** 0110001111100001 (0x63E1 / 25569)
- **Vulnerability Class:** Hardware Trojan (Combinational Trigger, Direct Key Payload Exfiltration)

—

2 VHDL Source Screenshots

The initial file structure and module source codes given in the challenge are depicted in Figure 1.

—

3 Technical Logic Analysis

3.1 Backdoor Evaluation Unit (backdoor.vhdl)

The backdoor block functions as a 16-bit equality comparator monitoring the input bus $D[15 : 0]$.

$$B = \begin{cases} 1 & \text{if } D[15 : 0] = \text{"1111111111101001"} \text{ (0xFFE9)} \\ 0 & \text{otherwise} \end{cases}$$

3.2 Top-Level Multiplexing (tpm.vhdl)

The signal B acts as a select control line for a 2-to-1 multiplexer driving the top-level outputs:

$$\text{Output}[15 : 0] = \begin{cases} K[15 : 0] & \text{if } B = 1 \text{ (Payload Leakage Mode)} \\ E[15 : 0] & \text{if } B = 0 \text{ (Standard Encryption Mode)} \end{cases}$$

When $D = 0xFFE9$, B switches to '1', directly exposing the hardcoded 16-bit key secret $K[15 : 0]$ onto the output lines.

```

[raviydv@parrot]~/Desktop/ctf/hardware_its_oops_pm
$ cat backdoor.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity backdoor is
    Port (
        D : in STD_LOGIC_VECTOR(15 downto 0);
        B : out STD_LOGIC
    );
end backdoor;

architecture Behavioral of backdoor is
    constant pattern : STD_LOGIC_VECTOR(15 downto 0) := "1111111111101001";
begin
    process(D)
    begin
        if D = pattern then
            B <= '1';
        else
            B <= '0';
        end if;
    end process;
end Behavioral;

```

(a) backdoor.vhdl

```

[raviydv@parrot]~/Desktop/ctf/hardware_its_oops_pm
$ cat encryption.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity encryption is
    Port (
        D, K : in STD_LOGIC_VECTOR(15 downto 0);
        E : out STD_LOGIC_VECTOR(15 downto 0)
    );
end encryption;

architecture Behavioral of encryption is
begin
    process(D, K)
    begin
        for i in 1 to 15 loop
            E(i) <= D(i) XOR K(i);
        end loop;

        E(0) <= NOT K(0);
        E(6) <= NOT K(6);
        E(13) <= NOT K(13);
    end process;
end Behavioral;

```

(b) encryption.vhdl

```

[raviydv@parrot]~/Desktop/ctf/hardware_its_oops_pm
$ cat key.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ckey is
    Port (
        K : out STD_LOGIC_VECTOR(15 downto 0)
    );
end ckey;

architecture Behavioral of ckey is
    constant key : STD_LOGIC_VECTOR(15 downto 0) := "0110001111100001";
begin
    K <= key;
end Behavioral;

[raviydv@parrot]~/Desktop/ctf/hardware_its_oops_pm
$ cat tpm.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tpm is
    Port (
        Data : in STD_LOGIC_VECTOR(15 downto 0);
        Output : out STD_LOGIC_VECTOR(15 downto 0)
    );
end tpm;

architecture Behavioral of tpm is
    signal Key : STD_LOGIC_VECTOR(15 downto 0);
    signal Encrypted : STD_LOGIC_VECTOR(15 downto 0);
    signal B : STD_LOGIC;

    component ckey
    port (
        K : out STD_LOGIC_VECTOR(15 downto 0)
    );
    end component;

```

(c) key.vhdl

```

component ckey
port (
    K : out STD_LOGIC_VECTOR(15 downto 0)
);
end component;

component encryption
port (
    D, K : in STD_LOGIC_VECTOR(15 downto 0);
    E : out STD_LOGIC_VECTOR(15 downto 0)
);
end component;

component backdoor
port (
    D : in STD_LOGIC_VECTOR(15 downto 0);
    B : out STD_LOGIC
);
end component;

begin
    ck : ckey port map(Key);
    enc: encryption port map (Data, Key, Encrypted);
    bd: backdoor port map (Data, B);

    process(Key, Encrypted, B)
    begin
        case B is
            when '1' =>
                for i in 0 to 15 loop
                    Output(i) <= Key(i);
                end loop;
            when others =>
                for i in 0 to 15 loop
                    Output(i) <= Encrypted(i);
                end loop;
        end case;
    end process;
end Behavioral;

```

(d) tpm.vhdl

Figure 1: VHDL Source Files Overview

3.3 Encryption Component Flaws (encryption.vhdl)

During standard operations ($B = 0$), the core performs bitwise XOR encryption across most bits, but intentionally forces outputs at bit positions 0, 6, and 13 to static inverted key values:

$$E(i) = D(i) \oplus K(i) \quad \text{for } 1 \leq i \leq 15$$

$$E(0) = \neg K(0), \quad E(6) = \neg K(6), \quad E(13) = \neg K(13)$$

4 Flag Output & Result Verification

Figure 2 captures the recovered flag and hardware output obtained after applying the backdoor activation vector.

```

[raviydv@parrot]~/Desktop/ctf/hardware_its_oops_pm
$xdg-open schematic.png
[raviydv@parrot]~/Desktop/ctf/hardware_its_oops_pm
$nc 154.57.164.82 30502
The input must be a binary signal of 16 bits.
Input : 111111111101001
Output: 0110001111100001
You triggered the backdoor here is the flag: HTB{4_7yp1c41_53cu23_TPM_ch1p}

```

Figure 2: Output Verification and Recovered Flag (vhd1flag.png)

Target Component	Binary Vector	Hexadecimal	Decimal
Backdoor Trigger Code	1111111111101001	0xFFE9	65513
Extracted Secret Key	0110001111100001	0x63E1	25569

Table 1: Input Trigger vs. Recovered Secret Key Data

5 Full VHDL Source Code Listings

5.1 backdoor.vhdl

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity backdoor is
6     Port (
7         D : in STD_LOGIC_VECTOR(15 downto 0);
8         B : out STD_LOGIC
9     );
10 end backdoor;
11
12 architecture Behavioral of backdoor is
13     constant pattern : STD_LOGIC_VECTOR(15 downto 0) := "1111111111101001";
14 begin
15     process(D)
16     begin
17         if D = pattern then
18             B <= '1';
19         else
20             B <= '0';
21         end if;
22     end process;
23 end Behavioral;

```

Listing 1: Backdoor Trojan Trigger Module

5.2 encryption.vhdl

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity encryption is
6     Port (
7         D, K : in STD_LOGIC_VECTOR(15 downto 0);
8         E : out STD_LOGIC_VECTOR(15 downto 0)
9     );

```

```

9      );
10 end encryption;
11
12 architecture Behavioral of encryption is
13 begin
14     process(D, K)
15     begin
16         for i in 1 to 15 loop
17             E(i) <= D(i) XOR K(i);
18         end loop;
19
20         E(0) <= NOT K(0);
21         E(6) <= NOT K(6);
22         E(13) <= NOT K(13);
23     end process;
24 end Behavioral;

```

Listing 2: Encryption Core Module

5.3 key.vhdl

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity ckey is
6     Port (
7         K : out STD_LOGIC_VECTOR(15 downto 0)
8     );
9 end ckey;
10
11 architecture Behavioral of ckey is
12     constant key : STD_LOGIC_VECTOR(15 downto 0) := "0110001111100001";
13 begin
14     K <= key;
15 end Behavioral;

```

Listing 3: Secret Key Storage Module

5.4 tpm.vhdl

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity tpm is
6     Port (
7         Data : in STD_LOGIC_VECTOR(15 downto 0);
8         Output : out STD_LOGIC_VECTOR(15 downto 0)
9     );
10 end tpm;
11
12 architecture Behavioral of tpm is
13     signal Key : STD_LOGIC_VECTOR(15 downto 0);
14     signal Encrypted : STD_LOGIC_VECTOR(15 downto 0);
15     signal B: STD_LOGIC;
16
17     component ckey
18         port ( K : out STD_LOGIC_VECTOR(15 downto 0) );
19     end component;
20

```

```

21  component encryption
22      port (
23          D, K : in STD_LOGIC_VECTOR(15 downto 0);
24          E : out STD_LOGIC_VECTOR(15 downto 0)
25      );
26  end component;
27
28  component backdoor
29      port (
30          D : in STD_LOGIC_VECTOR(15 downto 0);
31          B : out STD_LOGIC
32      );
33  end component;
34
35  begin
36      ck : ckey port map(Key);
37      enc: encryption port map (Data, Key, Encrypted);
38      bd: backdoor port map (Data, B);
39
40      process(Key, Encrypted, B)
41      begin
42          case B is
43              when '1' =>
44                  for i in 0 to 15 loop
45                      Output(i) <= Key(i);
46                  end loop;
47              when others =>
48                  for i in 0 to 15 loop
49                      Output(i) <= Encrypted(i);
50                  end loop;
51          end case;
52      end process;
53  end Behavioral;

```

Listing 4: Top-Level TPM Integration Module